

CloudFront Custom Error Pages for User-Friendly Failover Messages

To Begin with the Lab

Summary of the Lab

In this lab, an **Amazon CloudFront** distribution was configured with an **Application Load Balancer** as the primary origin and an **Amazon S3** static website as the backup error page source. A web server was launched on **Amazon EC2** and connected to the load balancer so the website could be accessed normally through CloudFront. To demonstrate the problem, the HTTP rule on the web server security group was removed, which caused the load balancer and CloudFront to return a 504 error. Instead of showing the default technical error, CloudFront was configured with a custom error response that pointed users to a friendly S3-hosted error page. Finally, the setup was verified by restoring and removing the HTTP rule to confirm that CloudFront switched between the normal website and the custom error page as expected.

Understanding Custom Error Pages in Amazon CloudFront

Amazon CloudFront custom error pages are used to replace technical error messages with a more user-friendly page when the origin fails. This is helpful when the backend application or load balancer is temporarily unavailable, because many users do not understand errors like 504 Gateway Timeout. Instead of showing a blank failure screen, CloudFront can redirect the viewer to a backup page hosted in **Amazon S3**. This exists to improve the user experience and help businesses communicate outages more clearly.

Example:

A website is served through an **Application Load Balancer**, but the backend **Amazon EC2** web server stops accepting traffic on port 80. CloudFront returns a 504 error by default, but a custom error page hosted in **Amazon S3** can display a message such as “Oops, something went wrong” so visitors know the site is temporarily unavailable.

Lab Steps

Step 1 — Create and Configure the EC2 Web Server

- Create an **Amazon EC2** instance and configure name as **web-server** and enable **Allow HTTP traffic from the internet**.

EC2 > Instances > Launch an instance

Name and tags Info

Name: Add additional tags

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search: Search our full catalog including 1000s of application and OS images

Recents | My AMIs | **Quick Start**

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
ami-0db56f446d44f2f09 (64-bit (x86), uefi-preferred) / ami-07c18f602ca3306b9 (64-bit (Arm), uefi)

Summary

Number of instances: 1

Firewall (security group): New security group

Storage (volumes): 1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier.

Cancel Launch instance Preview code

- Now we will configure it as a web server so we first must access the instance.
- For this, we will use **Command Prompt** and run the following commands.
- Now type: `cd {location_of_your_security_key}`.
- Now type: `ssh -i {security_key_name} ec2-user@{public_ipv4_address_of_ec2_instance}`.

```
ec2-user@ip-172-31-5-100:~
C:\Users\Shashank>cd downloads
C:\Users\Shashank\Downloads>ssh -i June-2026.pem ec2-user@65.2.73.108
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
[ec2-user@ip-172-31-5-100 ~]$
```

- Now login as root user by typing this command: `sudo -i`.
- Use `yum install httpd` command to configure and place webpage in Apache on the EC2 instance.

```
root@ip-172-31-6-87:~  
[ec2-user@ip-172-31-6-87 ~]$ sudo -i  
[root@ip-172-31-6-87 ~]# yum install httpd  
Amazon Linux 2023 Kernel Livepatch Repository 392 kB/s | 47 kB 00:00  
Last metadata expiration check: 0:00:01 ago on Fri Jun 12 09:07:49 2026.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
Installing:				
httpd	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	46 k
Installing dependencies:				
apr	x86_64	1.7.5-1.amzn2023.0.4	amazonlinux	129 k
apr-util	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	97 k
apr-util-ldb	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	13 k
generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3	amazonlinux	19 k
httpd-core	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	1.4 M
httpd-filesystem	noarch	2.4.67-1.amzn2023.0.1	amazonlinux	12 k
httpd-tools	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	80 k
libbrotli	x86_64	1.0.9-4.amzn2023.0.2	amazonlinux	315 k
mailcap	noarch	2.1.49-3.amzn2023.0.3	amazonlinux	33 k
Installing weak dependencies:				
apr-util-openssl	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	15 k
mod_http2	x86_64	2.0.39-1.amzn2023.0.1	amazonlinux	167 k
mod_lua	x86_64	2.4.67-1.amzn2023.0.1	amazonlinux	59 k

```
Transaction Summary  
=====
```

Package	Architecture	Version	Repository	Size
Install 13 Packages				
Total download size: 2.4 M				

- Replace the default web page with the provided index.html content with the help of vi editor.

```
Installed:  
apr-1.7.5-1.amzn2023.0.4.x86_64  
apr-util-ldb-1.6.3-1.amzn2023.0.2.x86_64  
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  
httpd-core-2.4.67-1.amzn2023.0.1.x86_64  
httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch  
httpd-tools-2.4.67-1.amzn2023.0.1.x86_64  
mailcap-2.1.49-3.amzn2023.0.3.noarch  
mod_lua-2.4.67-1.amzn2023.0.1.x86_64  
apr-util-1.6.3-1.amzn2023.0.2.x86_64  
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64  
httpd-2.4.67-1.amzn2023.0.1.x86_64  
libbrotli-1.0.9-4.amzn2023.0.2.x86_64  
mod_http2-2.0.39-1.amzn2023.0.1.x86_64  
Completed  
[root@ip-172-31-5-100 ~]# cd /var/www/html/  
[root@ip-172-31-5-100 html]# vi index.html
```

- Use the above commands to change content of the default index.html and paste your code in terminal.
- Now enable and run the web server services using these commands

```
systemctl start httpd  
systemctl enable httpd
```

- Verify that the EC2 website opens correctly using the public IP.

Step 2 — Create the CloudFront Distribution with the Load Balancer

- Open Amazon CloudFront.
- Click **Create distribution**.
- Select the **Application Load Balancer** as the origin.
- Create the **Application Load Balancer** with default settings.

Step 1 Choose a plan

Step 2 Get started

Step 3 **Specify origin**

Step 4 Enable security

Step 5 Review and create

Specify origin

Your origin is where your content (such as a website or app) lives. CloudFront works with AWS-based origins and origins hosted on other cloud providers.

Origin type

☐ Amazon S3
Deliver static assets like files and images, statically generated websites or single page applications (SPA).

☒ **Elastic Load Balancer**
Deliver applications hosted behind ELB such as dynamic websites, web services, and APIs.

☐ API Gateway
Deliver API endpoints for REST APIs hosted on API Gateway.

☐ Elemental MediaPackage
Deliver end-to-end live events or video on demand (VOD).

☐ VPC origin
Deliver applications and content hosted within private VPCs, such as EC2 instances and Application Load Balancers.

☐ Other
Refer to any AWS or non-AWS origin through its publicly resolvable URL.

Origin

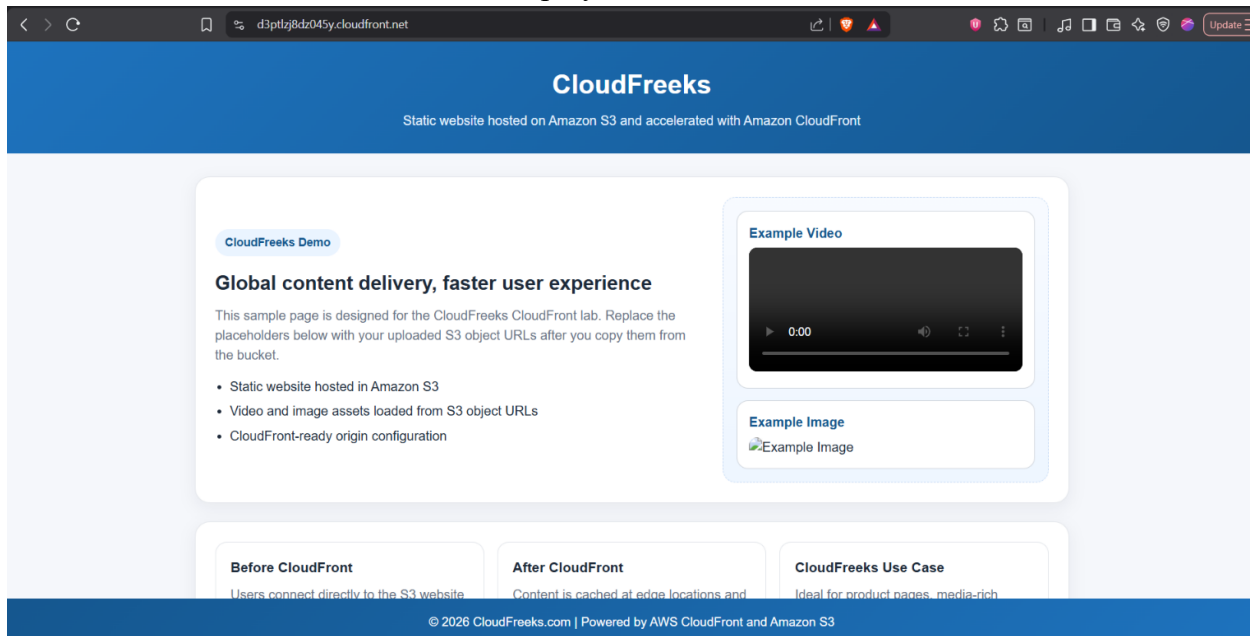
Elastic Load Balancing origin
Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

ALB-India-1346212286-ap-south-1.elb.amazonaws.com [Browse load balancers](#)

Origin path - optional
The directory path within your origin where your content is stored. [Learn more](#)

/path

- Keep the origin protocol as **HTTP**.
- Disable cache for testing so failover behavior is visible immediately.
- Do not enable **Web Application Firewall**.
- Create the distribution and wait for deployment.



- Verify that the website loads successfully from the CloudFront URL.

Step 3 — Create the S3 Backup Error Page

- Open **Amazon S3**.
- Create a new folder inside bucket for the error page.

Amazon S3

- Buckets**
 - General purpose buckets
 - Directory buckets
 - Table buckets
 - Vector buckets
- Files**
 - File systems
- Access management and security**
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer
- Storage management and insights**
 - Storage Lens
 - Batch Operations

Account and organization settings
AWS Marketplace for S3

Create folder

Your bucket policy might block folder creation
If your bucket policy prevents uploading objects without specific tags, metadata, or access control list (ACL) grantees, you will not be able to create a folder using this configuration. Instead, you can use the [upload configuration](#) to upload an empty folder and specify the appropriate settings.

Folder

Folder name
error /

Folder names can't contain "/" - [See rules for naming](#)

Server-side encryption Info
Server-side encryption protects data at rest.

The following encryption settings apply only to the folder object and not to sub-folder objects.

Server-side encryption

- ☒ **Don't specify an encryption key**
The bucket settings for default encryption are used to encrypt the folder object when storing it in Amazon S3.
- ☐ **Specify an encryption key**
The specified encryption key is used to encrypt the folder object before storing it in Amazon S3.

If your bucket policy requires objects to be encrypted with a specific encryption key, you must specify the same encryption key when you create a folder. Otherwise, folder creation will fail.

[Cancel](#) [Create folder](#)

- Make sure you have one static website hosting bucket as well for this lab.
- Upload an index.html file into error folder for the error page in the bucket.
- Enable **Static website hosting** for the bucket.

Amazon S3

- Buckets**
 - General purpose buckets
 - Directory buckets
 - Table buckets
 - Vector buckets
- Files**
 - File systems
- Access management and security**
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer
- Storage management and insights**
 - Storage Lens
 - Batch Operations

Account and organization settings
AWS Marketplace for S3

Edit static website hosting Info

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting

- ☐ Disable
- ☒ **Enable**

Hosting type

- ☒ **Host a static website**
Use the bucket endpoint as the web address. [Learn more](#)
- ☐ Redirect requests for an object
Redirect requests to another bucket or domain. [Learn more](#)

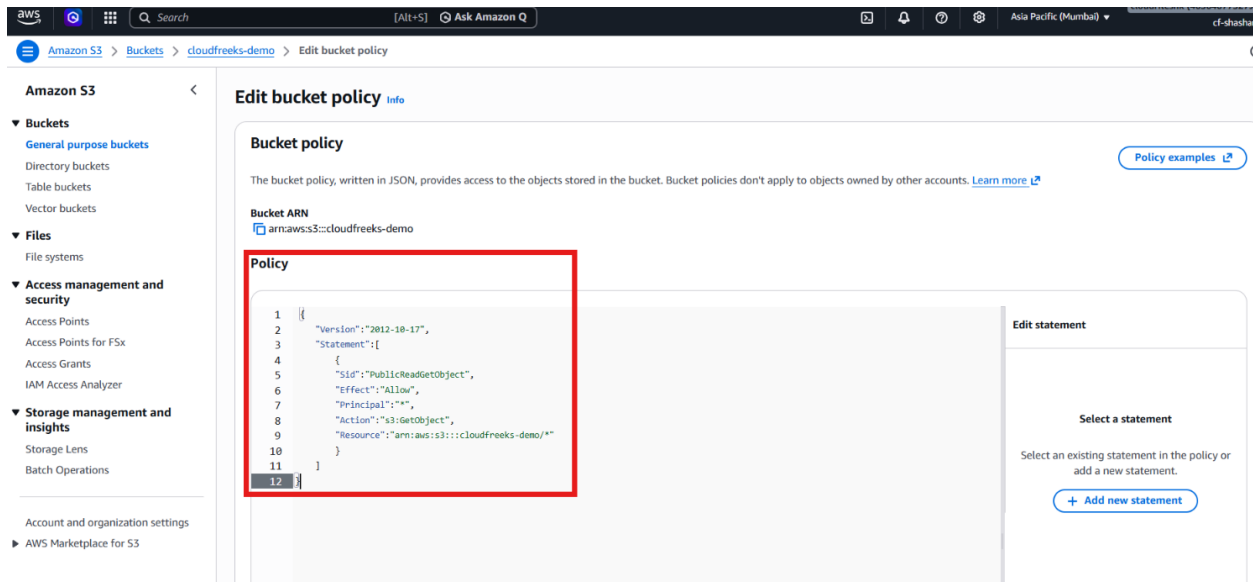
For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document
Specify the home or default page of the website.
index.html

Error document - optional
This is returned when an error occurs.
error.html

Redirection rules - optional
Redirection rules, written in JSON, automatically redirect webpage requests for specific content. [Learn more](#)

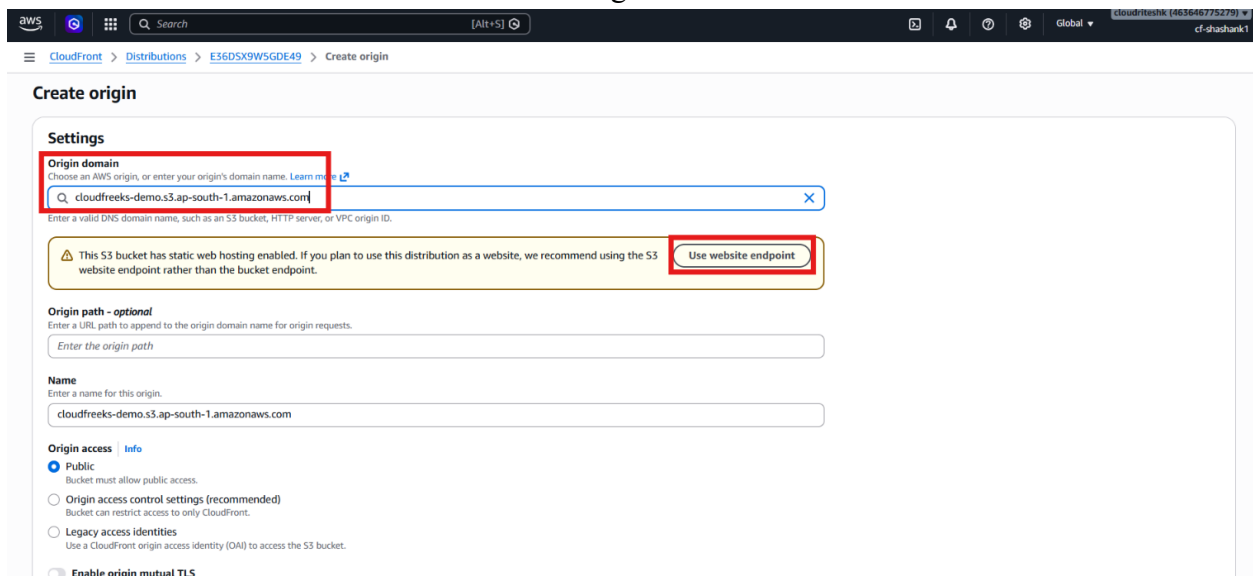
- Make the bucket public using the bucket policy.



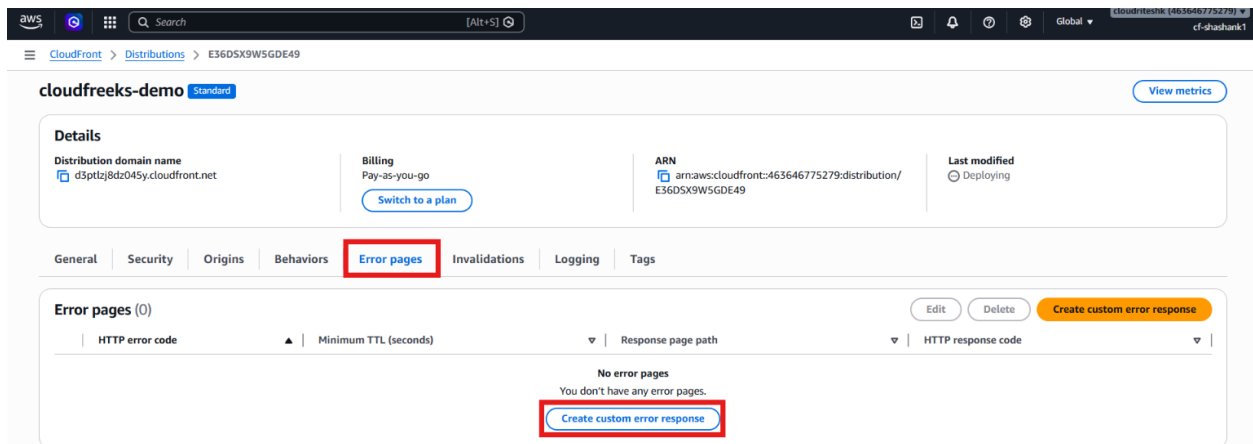
- Verify that the S3 static website endpoint opens correctly.
- Confirm that the page contains a user-friendly error message.

Step 4 — Configure the CloudFront Custom Error Response

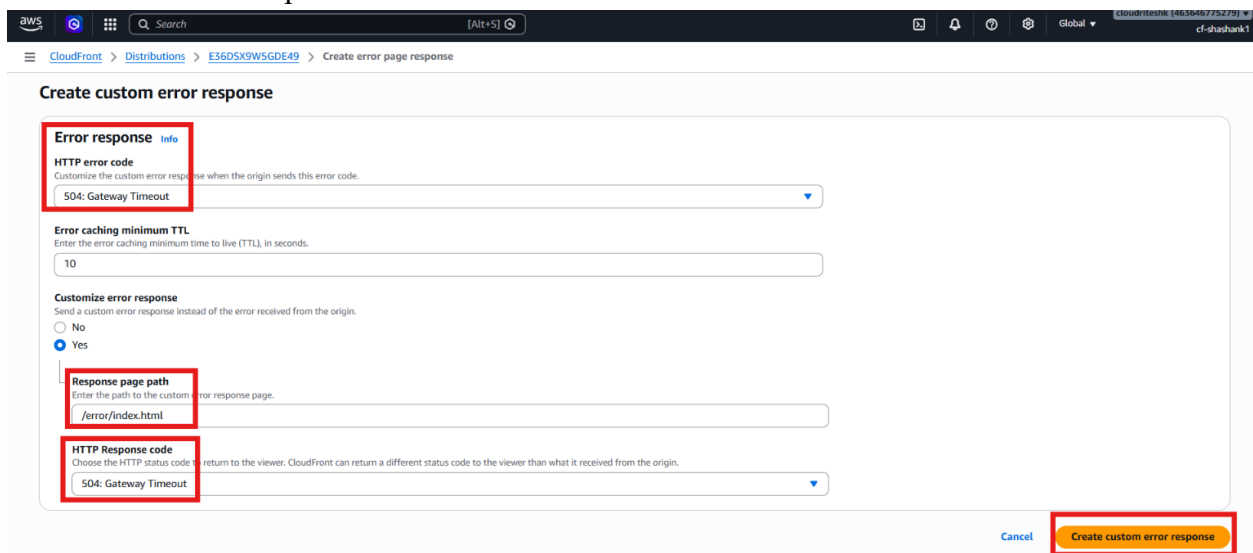
- Return to the CloudFront distribution.
- Add the **Amazon S3** bucket as a second origin.



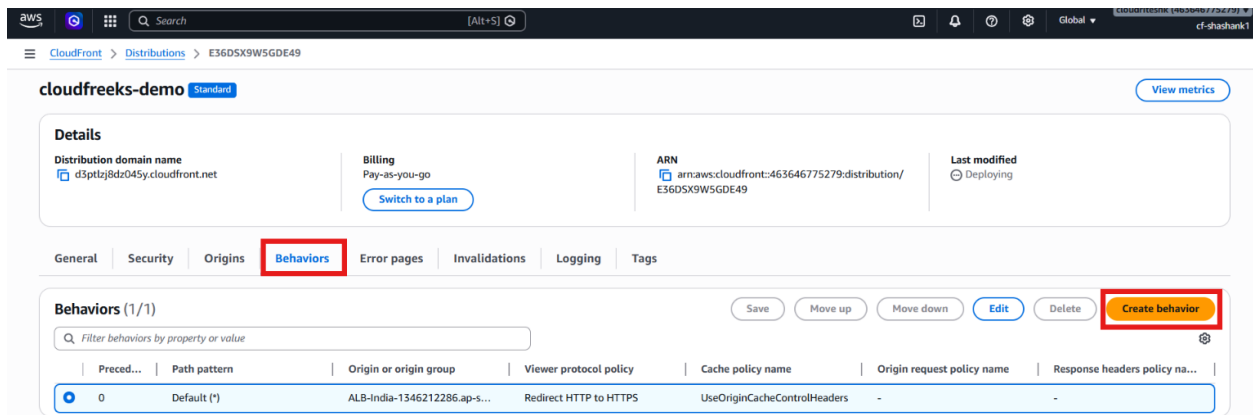
- Open Custom error responses.



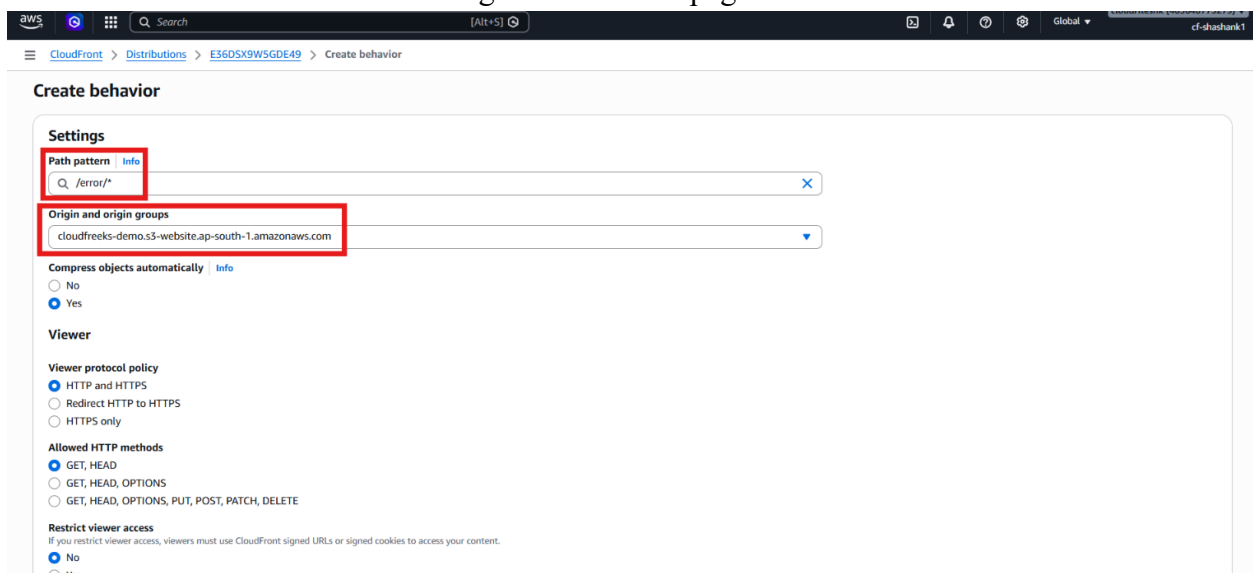
- Create a custom response for error code **504**.
- Point the error response to **error/index.html**.



- Add a behavior pattern for the error path.



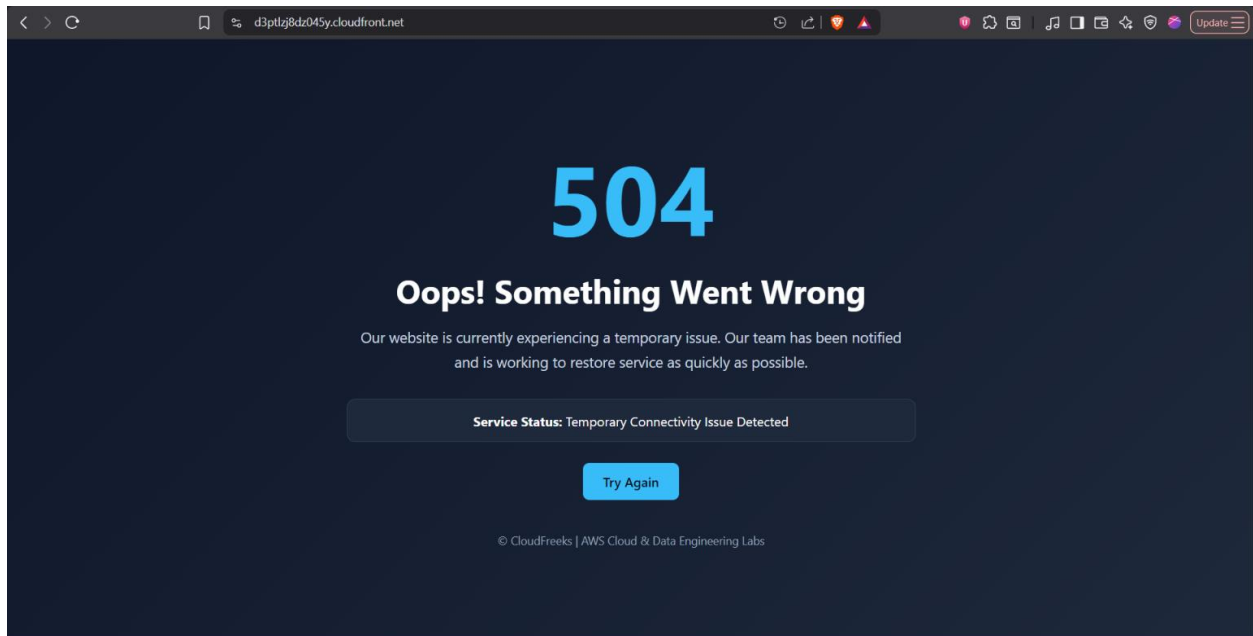
- Use the path as provided or custom if you have defined your own files.
- Set the behavior to use the S3 origin for the error page.



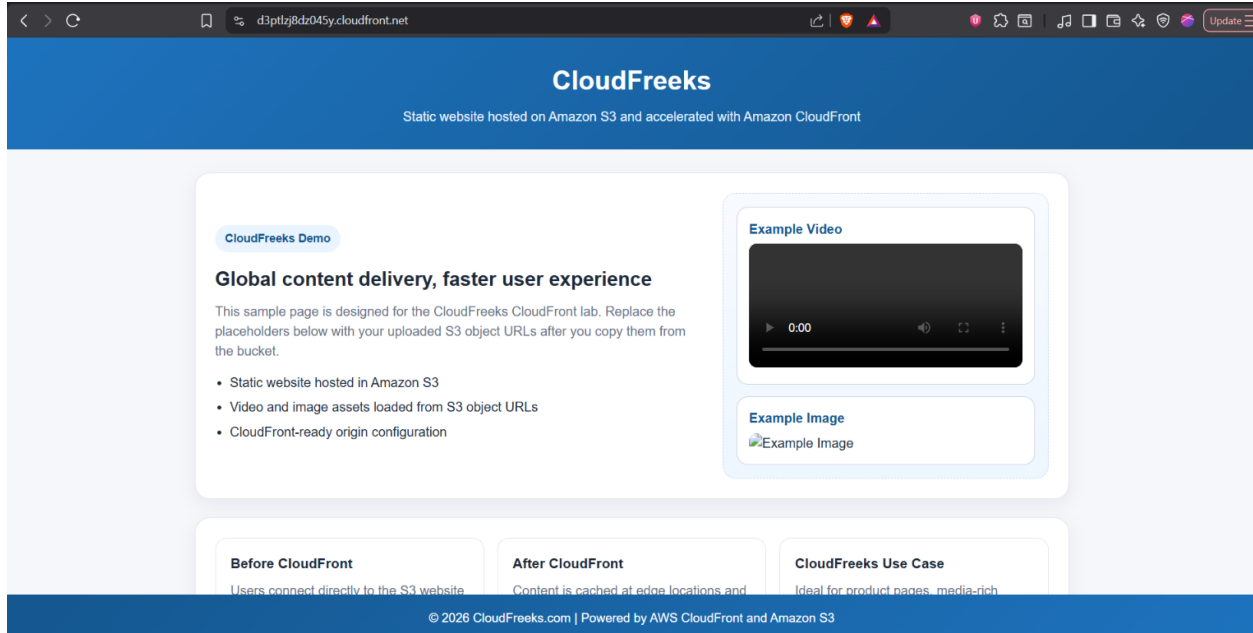
- Save the configuration and wait for deployment.

Step 5 — Test the Error Page and Recovery

- Return to **Amazon EC2**.
- Open the web server security group.
- Remove the **HTTP** inbound rule on port 80.
- Save the changes.
- Refresh the CloudFront URL.



- Verify that the default 504 error is replaced with the custom S3 error page.
- Restore the **HTTP** inbound rule on port 80.
- Refresh the CloudFront URL again.
- Verify that the normal website loads successfully.



- Confirm that CloudFront serves the custom error page when the origin fails and restores the website when the origin is healthy again.

Verification

- The EC2 web server was successfully configured behind an Application Load Balancer.
- CloudFront returned a user-friendly custom error page instead of a technical 504 error.

- The S3-hosted backup page loaded correctly during origin failure.
- Restoring the HTTP rule brought the website back to normal.
- CloudFront custom error responses worked as expected for user-friendly outage handling.